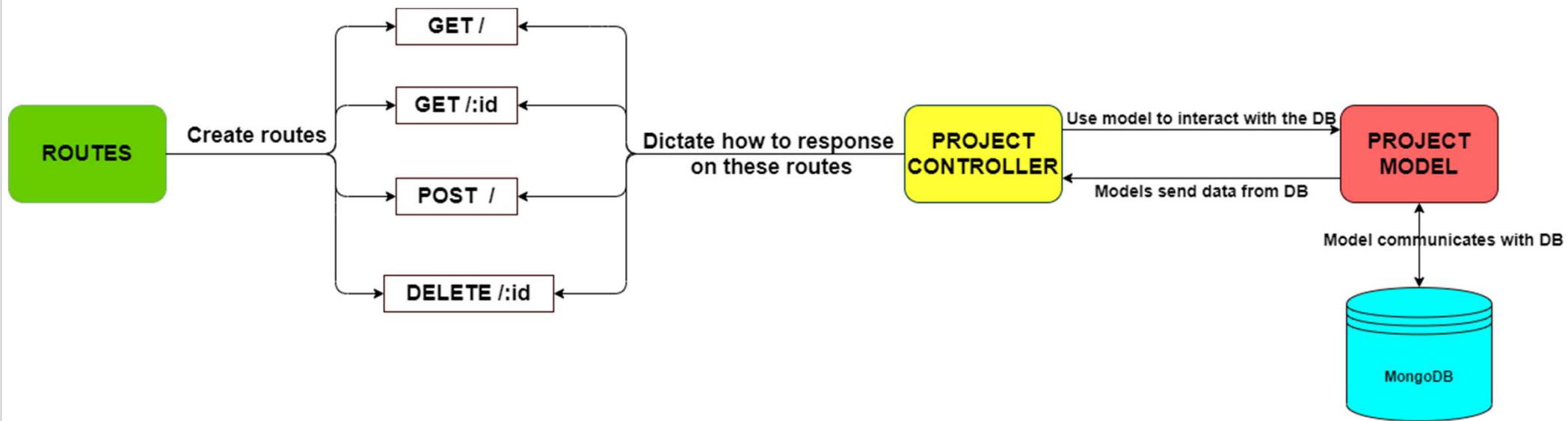




## The project architecture



## Building RESTful APIs with Express.js

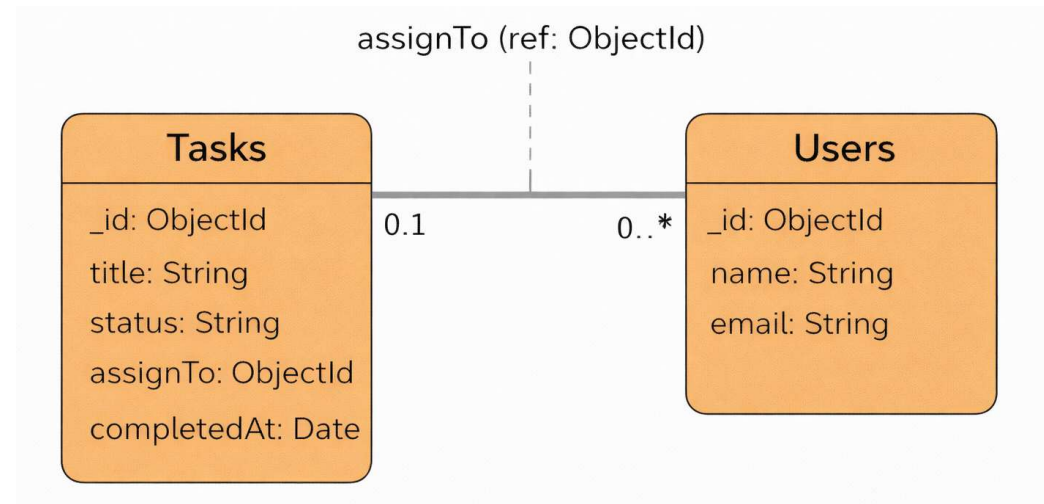
---

Express.js simplicity and flexibility make it a great option for creating RESTful APIs. We can build the RESTful API by doing the following steps:

- **Setup the Project:** Create a new Node.js project and install express.js.
- **Create Express App:** Require Express and create an instance using `express()`.
- **Define Routes:** Designate route handlers for HTTP methods (GET, POST, PUT, DELETE) and endpoints. These handlers will determine how the API responds to specific requests.
- **Middleware:** Implement middleware functions to handle tasks like authentication, request parsing, error handling, etc. Middleware functions are executed before reaching the route handlers.
- **Create Controllers:** Create controllers by separating the business logic from the route handlers. Controllers manage data processing and interact with databases or other services.
- **Connect to Database:** If your API requires data storage, connect to a database (e.g., MongoDB) using a suitable database driver.
- **Create HTML Pages with EJS:** using ejs to render HTML code, allowing us to view the code in the browser
- **Test API Endpoints:** Use tools like Postman or URL to test the API endpoints and verify they return the expected responses.

## Project Structure

- ✓ src
  - > config
  - ✓ controllers
    - JS taskController.js
    - JS userController.js
  - ✓ models
    - JS Task.js
    - JS User.js
  - ✓ routes
    - JS taskRoutes.js
    - JS userRoutes.js
  - ✓ views
    - <> createTask.ejs
    - <> listTask.ejs
- ⚙ .env
- 📁 .gitignore
- { } package-lock.json
- { } package.json
- JS server.js



## Step 1: Set Up Project

### ❑ Initialize a New Project

Create a new directory for your project. Then initialize the project with npm: **npm init -y**

### ❑ Or you can install [nodemon]

(<https://www.npmjs.com/package/nodemon>) for the same purpose. Nodemon detects changes in the directory and restarts your application.

```
npm install nodemon -D
```

### ❑ Then config in package.json:

```
"scripts": {  
  "dev": "nodemon app.js",  
  "start": "node app.js"  
}
```

### ❑ Then run: **npm run dev**

package.json

package.json > ...

```
1  {  
2    "name": "backend",  
3    "version": "1.0.0",  
4    "description": "",  
5    "main": "server.js",  
6    "type": "module",  
7    "scripts": {  
8      "dev": "nodemon server.js",  
9      "start": "node server.js"  
10   },  
11   "keywords": [],  
12   "author": "",  
13   "license": "ISC",  
14   "dependencies": {  
15     "express": "^5.1.0"  
16   },  
17   "devDependencies": {  
18     "nodemon": "^3.1.10"  
19   }  
20 }
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

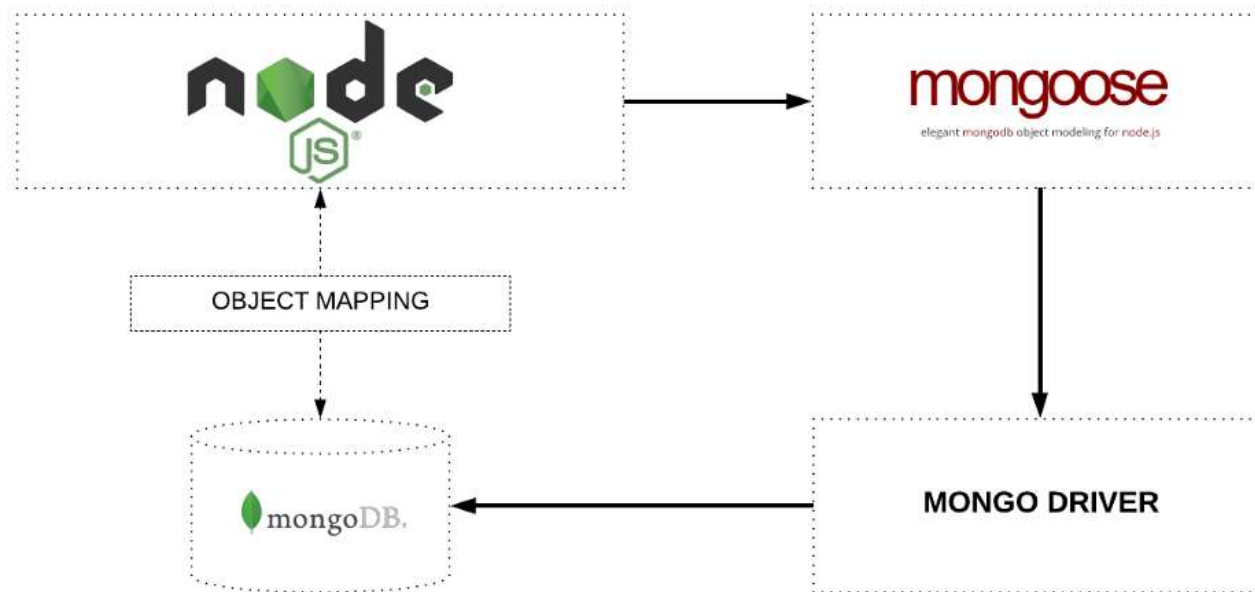
PORTS

PS D:\SDN302\taskmanager\backend> npm run dev

## Step 2: Connect to the Database

**Mongoose** is a JavaScript library that makes it easier to work with MongoDB in a Node.js environment. It provides a simple way to model your data. You can define schemas, do data validation, and build queries as well. Install Mongoose using npm.

```
npm i mongoose
```



# MongoDB Compass

CONNECTIONS (1)

Search connections

huanll

admin

config

local

taskmanager

tasks

users

Type a query: { field: 'value' } or [Generate query](#)

Explain

Reset

Find

Options

ADD DATA

UPDATE

DELETE

EXPORT DATA

EXPORT CODE

25 1 - 2 of 2

Task 1

Task 2

## New Connection

Manage your connection settings

URI

Edit Connection String ☒

mongodb://localhost:27017/

How do I find my connection string in Atlas?

If you have an Atlas cluster, go to the Cluster view. Click the

## Step 2: Connect to the Database

Create a **db.js** file inside **config** folder. You need to define the connection URL provided in your db.js file:

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    await mongoose.connect('mongodb://127.0.0.1:27017/taskmanager')
    console.log('Connection successful')
  } catch (error) {
    console.error("Error connecting to database:", error);
    process.exit(1); // exit with error
  }
}

module.exports = connectDB;
```

In your **server.js** file, you need to require Mongoose to use it as a connector to the database.

```
const connectDB = require('./src/config/db')

// Connect to the Database
connectDB();
```

## Step 2: Set up your .env and .gitignore files

we'll upload the project to GitHub. You'll protect your port number and MongoDB URL with secure storage. You'll also use an environment variable file, a configuration file known as .env. This file keeps sensitive information safe. It includes passwords and API URLs and prevents exposure. Here's how to set it up using Node.js:

```
npm i dotenv
```

Then create a .env file. Inside it, add your PORT number and MongoDB URL. It should look something like this:

```
PORT=5000  
MONGODB_URI=mongodb://127.0.0.1:27017/taskmanager
```

Then update your .gitignore file:

```
/node_modules  
.env
```

Then in your server.js file, require the dotenv package. Load the variables with this line at the top of the file:

```
const dotenv = require('dotenv');  
dotenv.config();
```

## Step 2: Set up your .env and .gitignore files

Then, in your Node.js code, you can access these variables using the `process.env` object:

In the `db.js` file edit the codes below:

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGODB_URI)
    console.log('Connection successful')
  } catch (error) {
    console.error("Error connecting to database:", error);
    process.exit(1); // exit with error
  }
}

module.exports = connectDB;
```

In the `server.js` file edit the codes below:

```
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Server is running on
http://localhost:${PORT}`)
})
```

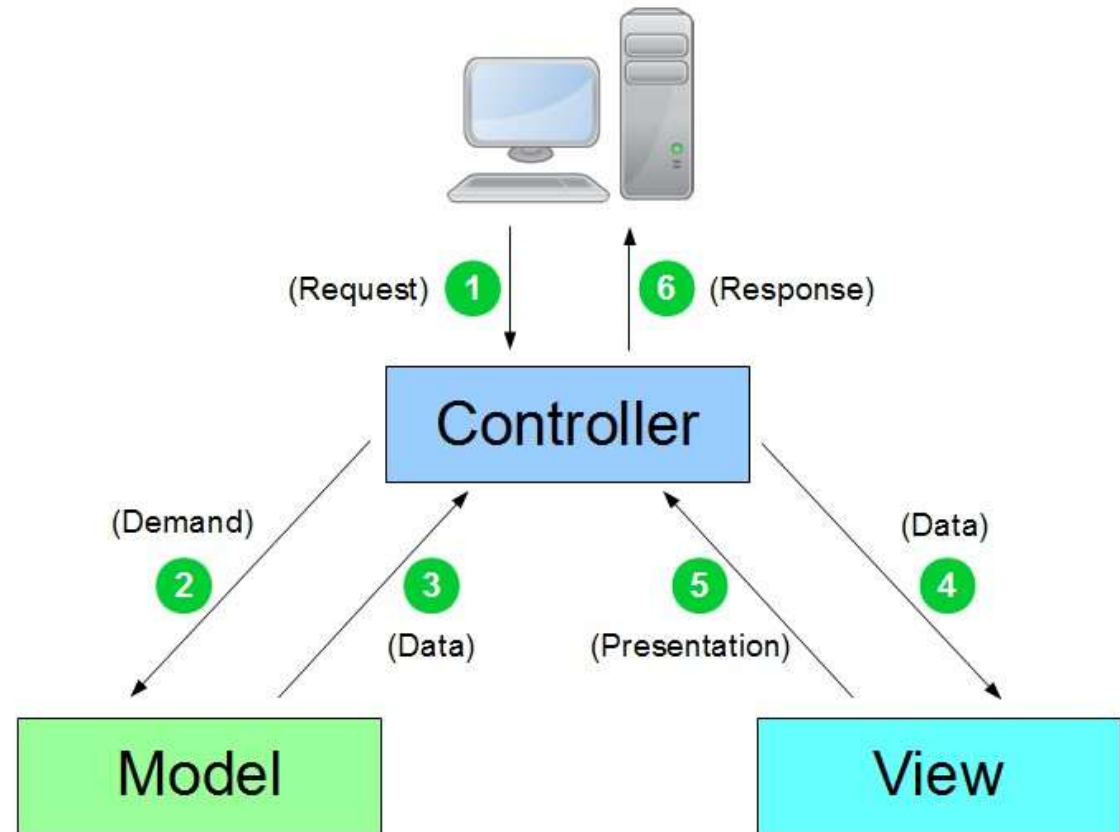
## Model/View/Controller structure

The MVC pattern divides an application into three distinct parts:

**1.Model:** Represents the data and business logic of the application. It interacts with the database and processes data independently of the user interface.

**2.View:** Handles the presentation layer of the application. It displays data to the user and sends user commands to the Controller.

**3.Controller:** Acts as an intermediary between the Model and the View. It takes user input, interacts with the Model, and updates the View accordingly.



## Step 3: Define Model

---

In the `src/models/User.js` file edit the codes below:

```
const mongoose = require('mongoose');
const userSchema = mongoose.Schema(
  {
    name: {
      type: String, required: true
    },
    email: {
      type: String, required: true, unique: true
    },
    createdAt: {
      type: Date, default: Date.now
    }
  }
)

const User = mongoose.model('User', userSchema);
module.exports = User;
```

## Step 3: Define Model

In the `src/models/Task.js` file edit the codes below:

```
const mongoose = require('mongoose');
const taskSchema = new mongoose.Schema(
  {
    title: {
      type: String, required: true, trim: true,
    },
    status: {
      type: String, enum: ["active", "complete"], default: "active",
    },
    completedAt: {
      type: Date, default: null,
    },
    assignTo: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User'
    }
  },
  {
    timestamps: true, // createdAt and updatedAt are automatically added
  }
);
const Task = mongoose.model("Task", taskSchema);
module.exports = Task;
```

## Step 4: Define Controller

In the `taskControllers.js` file edit the codes below:

```
const Task = require('../models/Task');
const User = require('../models/User')

// Render EJS
exports.getTasks = async (req, res) => {
  const tasks = await Task.find()
  // POPULATION
  .populate('assignTo', 'name email');
  res.render('listTask', { tasks });
};

exports.createTaskForm = async (req, res) => {
  const users = await User.find();
  res.render('createTask', { users });
};
```

```
// API for HTML
exports.createTask = async (req, res) => {
  await Task.create({
    title: req.body.title,
    assignTo: req.body.assignTo
  });
  res.redirect('/api/tasks/list');
};

exports.updateTaskStatus = async (req, res) => {
  await Task.findByIdAndUpdate(req.params.id, {
    status: 'complete',
    completedAt: new Date()
  });
  res.redirect('/api/tasks/list');
};

exports.deleteTask = async (req, res) => {
  await Task.findByIdAndDelete(req.params.id);
  res.redirect('/api/tasks/list');
}
```

## Step 4: Define Controller

In the `userControllers.js` file edit the codes below:

```
const User = require('../models/User')

exports.getAllUsers = async (req, res) => {
  try {
    const users = await User.find()
    res.status(200).json(users);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
}

exports.createUsers = async (req, res) => {
  try {
    const users = new User(req.body);
    await users.save();
    res.status(201).json(users);
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
};
```

## Step 5: Define Routes

---

In the `taskRoutes.js` file edit the codes below:

```
const express = require('express');
const ctrl = require('../controllers/taskControllers')
const router = express.Router();

// routing path postman
router.get('/', ctrl.getAllTasks);
router.post('/', ctrl.createTasks);
router.put('/:id', ctrl.updateTasks);
router.delete('/:id', ctrl.deleteTasks);

// routing path form
router.post('/create', ctrl.createTask);
router.post('/complete/:id', ctrl.updateTaskStatus);
router.post('/delete/:id', ctrl.deleteTask);

// routing path ejs
router.get('/list', ctrl.getTasks);
router.get('/create', ctrl.createTaskForm);
module.exports = router;
```

## Step 5: Define Routes

---

In the `userRoutes.js` file edit the codes below:

```
const express = require('express');
const ctrl =
require('../controllers/userControllers')
const router = express.Router();

// routing path
router.get('/', ctrl.getAllUsers);
router.post('/', ctrl.createUsers);

module.exports = router;
```

## Step 6: Set Up the Server.js

```
const express = require('express');
const tasksRoutes = require('./src/routes/taskRoutes');
const usersRoutes = require('./src/routes/userRoutes');
const connectDB = require('./src/config/db')
const path = require('path');
const dotenv = require('dotenv');
dotenv.config();
const app = express();

// middlewares
app.use(express.json());
// Parse form-urlencoded (form HTML)
app.use(express.urlencoded({ extended: true }));

// Set EJS as the template engine and views directory
app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'src', 'views'));

// Connect to the Database
connectDB();

app.use('/api/tasks', tasksRoutes);
app.use('/api/users', usersRoutes);

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`)
})
```

## Step 7: Create HTML Pages with EJS – listTask.ejs

```
<body>
  <h2>Task List</h2>
  <a href="/api/tasks/create">Create Task</a>
  <ul>
    <% tasks.forEach(task=> { %>
      <li>
        <strong>
          <%= task.title %>
        </strong>
        <form action="/api/tasks/complete/<%= task._id %>" method="POST">
          <button>Complete</button>
        </form>
        <form action="/api/tasks/delete/<%= task._id %>" method="POST">
          <button>Delete</button>
        </form>
        <br>
        Assigned to:
        <%= task.assignTo ? task.assignTo.name : 'Unassigned' %>
        (<%= task.assignTo ? task.assignTo.email : '' %>)
        <br>
        Status: <%= task.status %>
        <br>
        Completed: <%= task.completedAt %>
      </li>
    <% }) %>
  </ul>
</body>
```

```
▼ views
  <> createTask.ejs
  <> listTask.ejs
```

## Step 7: Create HTML Pages with EJS – createTask.ejs

---

```
<body>
  <h2>Create Task</h2>

  <form action="/api/tasks/create" method="POST">
    <input type="text" name="title" placeholder="Task title" required />

    <select name="assignTo">
      <% users.forEach(user=> { %>
        <option value="<%= user._id %>">
          <%= user.name %> - <%= user.email %>
        </option>
      <% }) %>
    </select>

    <button>Create</button>
  </form>
</body>
```

# DEMO

## 1. Test getAllUsers → Result: Empty

The screenshot shows a REST client interface with a sidebar on the left containing a collection of endpoints under 'TaskManager'. The main panel displays a GET request to 'http://localhost:5000/api/users/'. The response status is '200 OK' with a body of '[]' (empty array). A red arrow points to the '[]' with the word 'Empty' next to it.

TaskManager / Get User

GET http://localhost:5000/api/users/ Send

Docs Params Auth Headers (7) Body Scripts Tests Settings Cookies

Headers 7 hidden

| Key | Value | Des...      | Bulk Edit | Presets |
|-----|-------|-------------|-----------|---------|
| Key | Value | Description |           |         |

Body 200 OK • 9 ms • 235 B Save Response

{ } JSON Preview Visualize

1 []

Empty

# DEMO

## 2. Create User

The screenshot displays a REST client interface with a sidebar on the left containing a collection tree. The 'TaskManager' collection is expanded, and the 'POST Create User' endpoint is selected and highlighted with a red box. The main panel shows the request configuration for the 'Create User' endpoint, with the method 'POST' and the URL 'http://localhost:5000/api/users/' highlighted by a red box. The 'Body' tab is active, showing a JSON request body: 

```
{  "name": "Lam Le Huan",  "email": "lamlehuan@gmail.com"}
```

. Below the request, the response section shows a status of '201 Created' with a response body highlighted by a red box: 

```
{  "name": "Lam Le Huan",  "email": "lamlehuan@gmail.com",  "_id": "6976d707671eb8761768c9ce",  "createdAt": "2026-01-26T02:52:55.920Z",  "__v": 0}
```

TaskManager / Create User

POST http://localhost:5000/api/users/ Send

Docs Params Auth Headers (9) Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "name": "Lam Le Huan",
3   "email": "lamlehuan@gmail.com"
4 }
```

Body 201 Created 87 ms 373 B Save Response

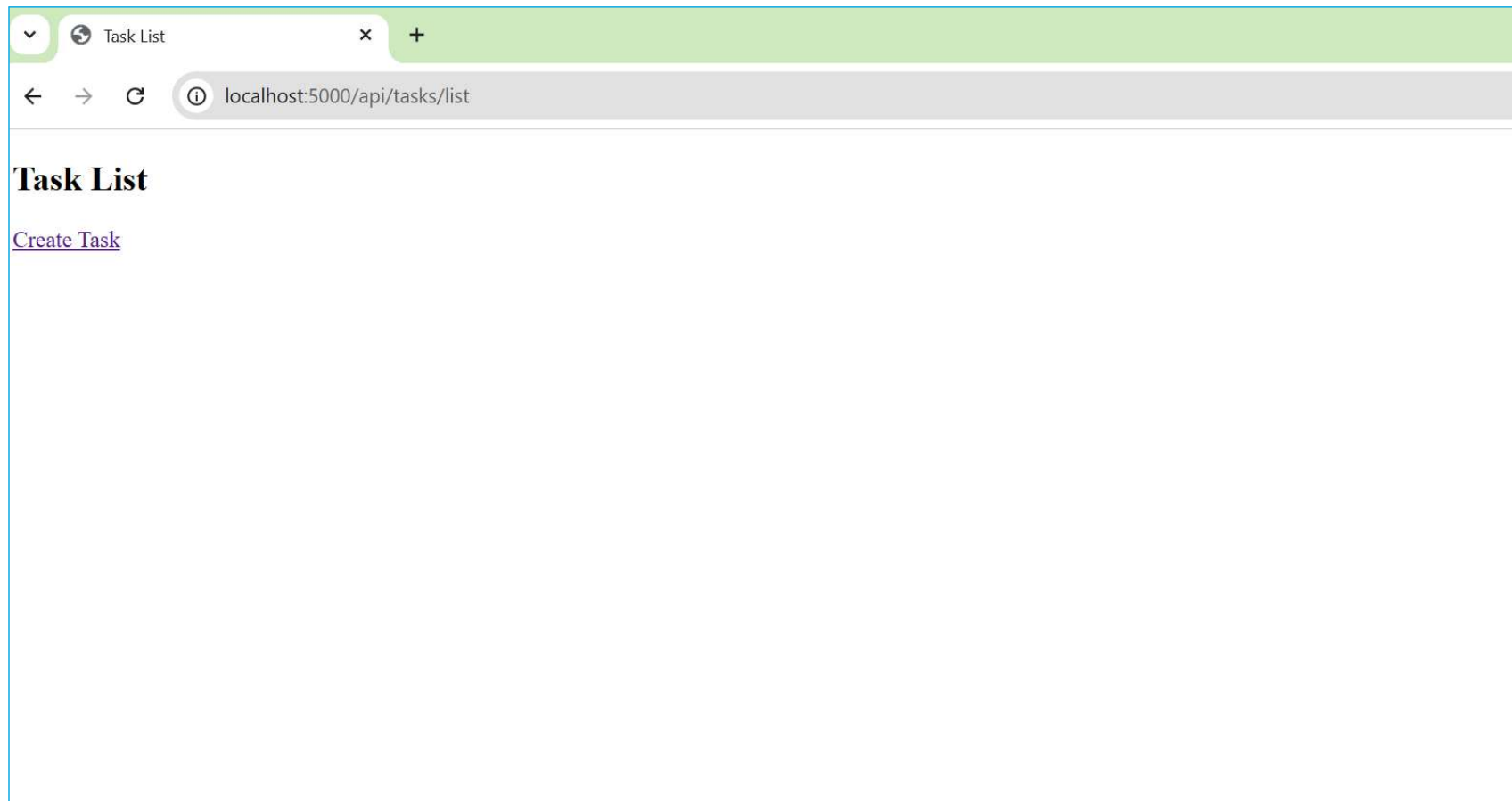
{ JSON Preview Visualize

```
1 {
2   "name": "Lam Le Huan",
3   "email": "lamlehuan@gmail.com",
4   "_id": "6976d707671eb8761768c9ce",
5   "createdAt": "2026-01-26T02:52:55.920Z",
6   "__v": 0
7 }
```

## DEMO

---

3. <http://localhost:5000/api/tasks/list>



## DEMO

### 4. Create new Task



▼ Create Task × +

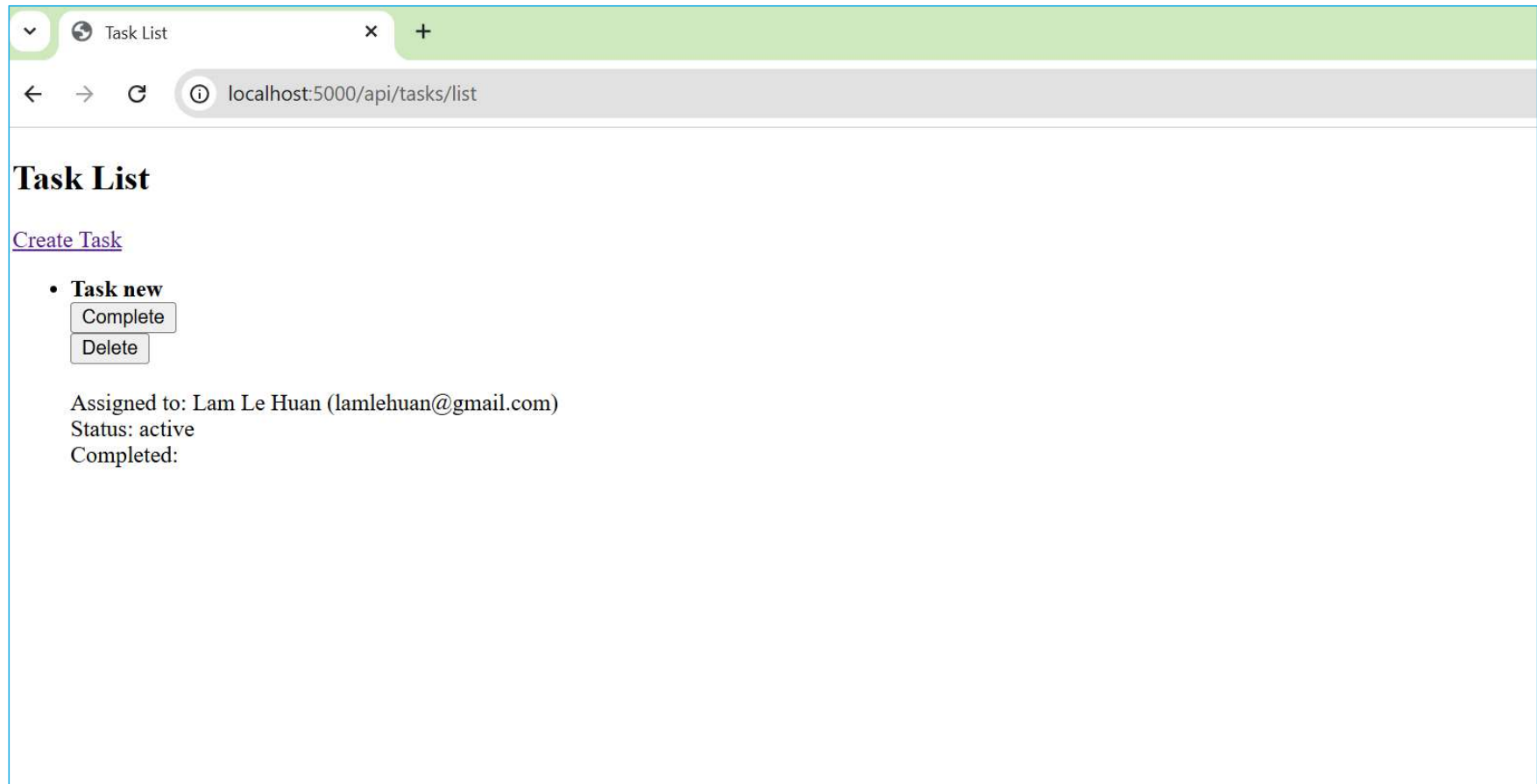
← → ↻ ⓘ localhost:5000/api/tasks/create

### Create Task

Task new Lam Le Huan - lamlehuan@gmail.com ▼ Create

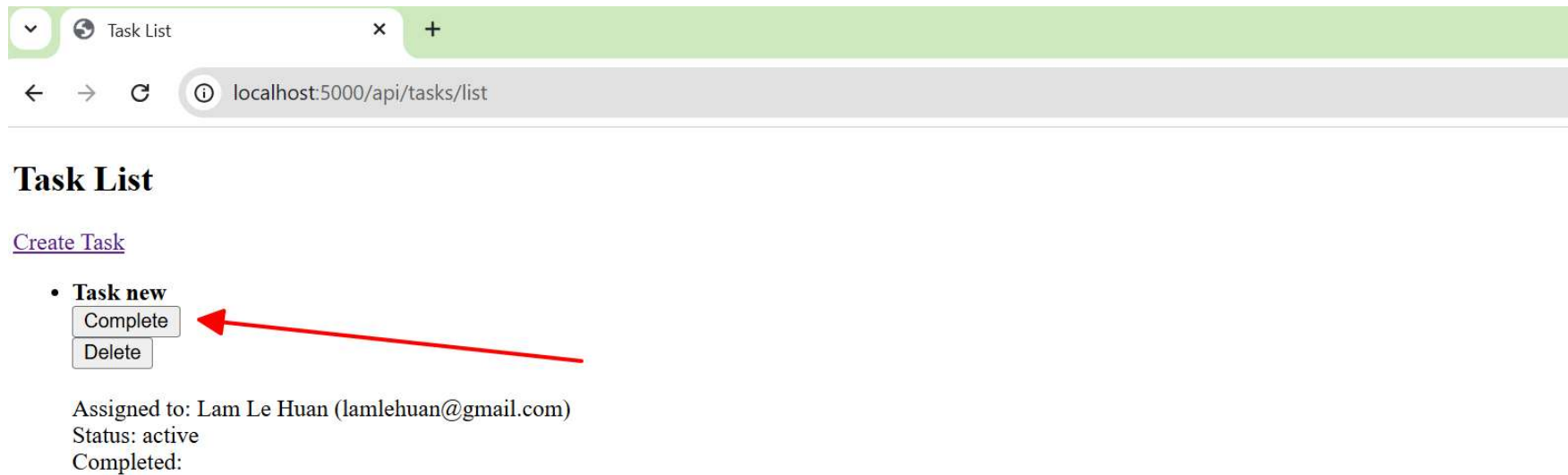
## DEMO

### 5. Show list task



## DEMO

### 6. Update Complete



Task List

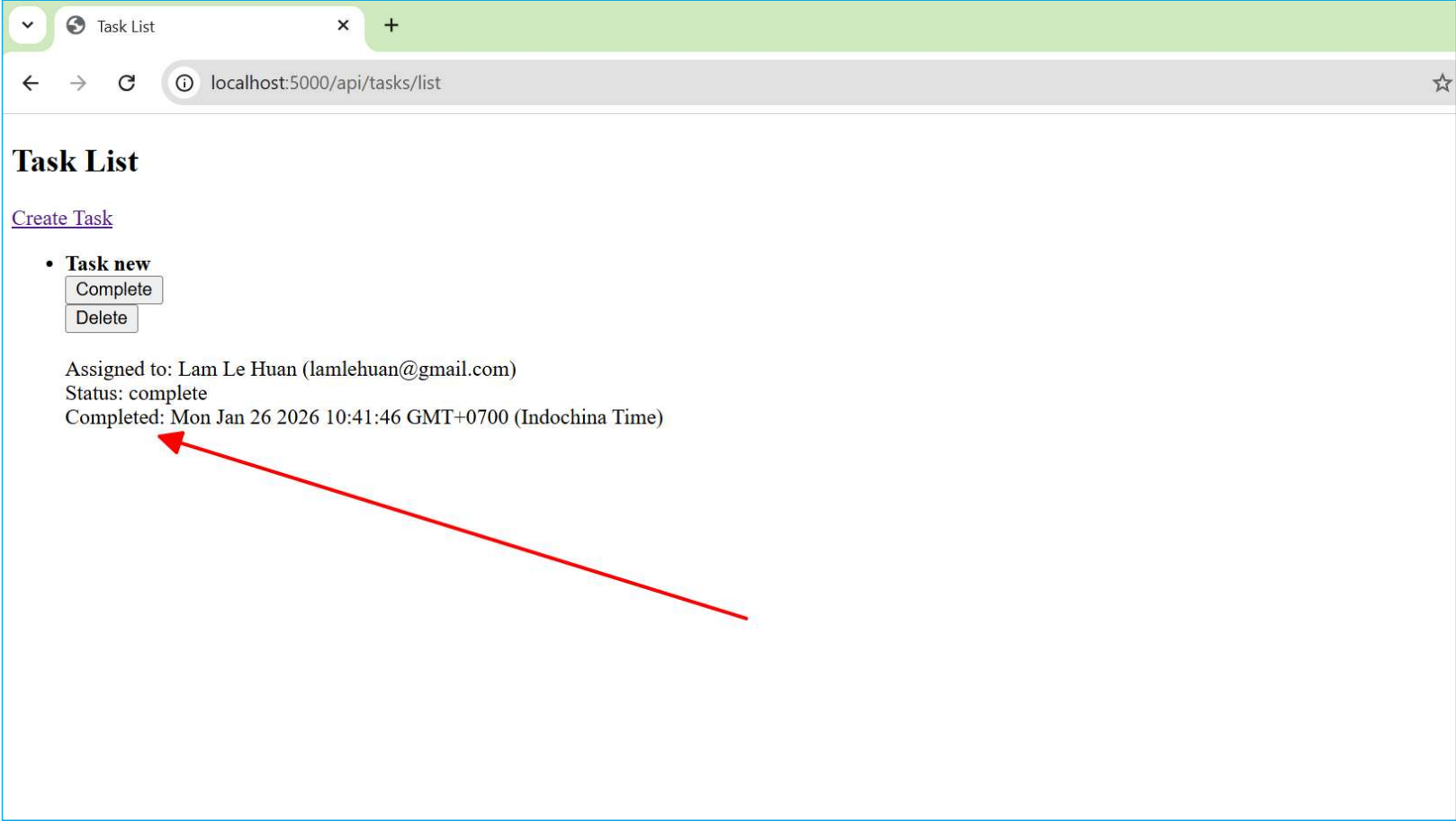
[Create Task](#)

- **Task new**
  - Complete
  - Delete

Assigned to: Lam Le Huan (lamlehuan@gmail.com)  
Status: active  
Completed:

## DEMO

### 7. Show task complete with date complete



The screenshot shows a web browser window with the title "Task List" and the address bar displaying "localhost:5000/api/tasks/list". The page content includes a "Task List" heading, a "Create Task" link, and a list item "Task new". This item has "Complete" and "Delete" buttons. Below the buttons, the task details are shown: "Assigned to: Lam Le Huan (lamlehuan@gmail.com)", "Status: complete", and "Completed: Mon Jan 26 2026 10:41:46 GMT+0700 (Indochina Time)". A red arrow points to the "Completed" date and time string.

**Task List**

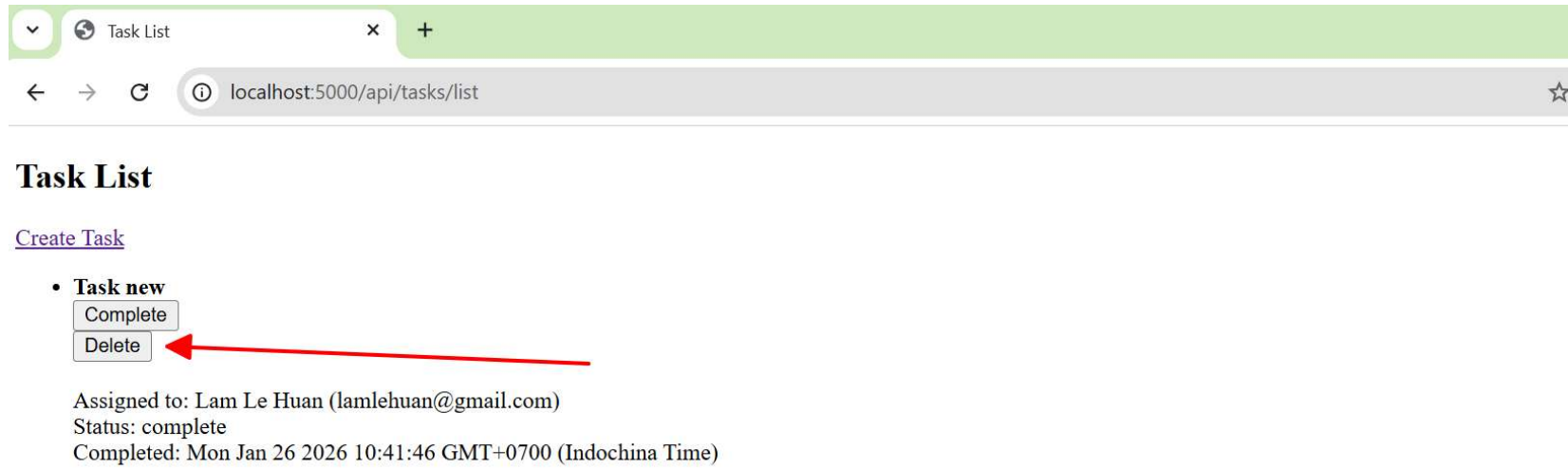
[Create Task](#)

- **Task new**  
Complete  
Delete

Assigned to: Lam Le Huan (lamlehuan@gmail.com)  
Status: complete  
Completed: Mon Jan 26 2026 10:41:46 GMT+0700 (Indochina Time)

## DEMO

### 8. Delete Task new



Task List

[Create Task](#)

- **Task new**  
Complete  
Delete

Assigned to: Lam Le Huan (lamlehuan@gmail.com)  
Status: complete  
Completed: Mon Jan 26 2026 10:41:46 GMT+0700 (Indochina Time)

## DEMO

---

### 9. Result list empty task

